Contents

1 Streaming Step **2**
1.1 Periodic Shift Streaming . . . 2
1.1.1 Basic idea . . . 2
1.1.2 Full PS algorithm . . . 3
1.1.3 Note about periodic boundary condition . . . 4
2 Boundary Conditions **5**
2.1 Moment based boundary conditions . . . 5
2.1.1 General procedure . . . 5
2.1.2 Corner cell example . . . 6
2.2 Reconstruction of macroscopic quantities from known distributions . . . 9
2.2.1 Corner cell example with unknown ρ, u_x, u_y, u_z . . . 9
2.2.2 Corner cell example with prescribed ρ or u_x, u_y, u_z . . . 12
3 Grid refinement **15**
3.1 Adjustment of parameters on the fine grid . . . 15
3.2 Communication on the grid interface . . . 17
4 References **20**

1 Streaming Step

During the streaming step, distributions are propagated into the neighbour cells in the corresponding direction. There are several ways to implement the streaming step in memory. The simplest approach is the A-B streaming pattern, which uses two distribution fields. Distributions are read from old locations in field A and written into new locations in field B. This doubles memory consumption.

To prevent the second field, it is necessary to find an algorithm that always writes data into the same locations where it read from. One of such algorithms is the A-A streaming, which combines streaming and collision steps together and alters between odd and even timesteps.

There is also another way that does not require altering between odd and even timesteps presented in [6], called Periodic Shift.

1.1 Periodic Shift Streaming

Article [6] presents the algorithm "Periodic Shift" (PS). This algorithm requires the "Structure of Arrays" (SoA) organization so that each distribution array $\underline{f_q}$ is saved separately in a continous block of memory. Instead of physically writing data into new positions during the streaming step, it uses periodic shifting of the memory index, independent for each distribution.

1.1.1 Basic idea

Before formalizing the algorithm, let's illustrate the idea on a D2Q9 scheme (Figure 1):

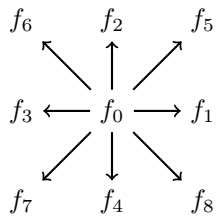

Figure 1: D2Q9 scheme

Consider a field with 3 rows and 4 columns with cells labelled using row-major indexing as shown in Figure 2:

0	1	2	3
4	5	6	7
8	9	10	11

Figure 2: D2Q9 field with cell labels

During streaming, values of each distribution array $\underline{f_q}$ move through the field in the direciton given by the velocity set. This means, after streaming, in cell $\overline{2}$ we want to find the original value of f_1 from cell 1, original value of f_2 from cell 6, etc. Values f_4, f_7, f_8 are unknown ("?") in cell 2 and need to be filled by a boundary condition. Table 1 shows, for every cell and every distribution f_q, from where the value was streamed.

cell i	0	1	2	3	4	5	6	7	8	9	10	11
f_q	**In cell** i**, where was the value of** f_q **streamed from?**											
f_0	0	1	2	3	4	5	6	7	8	9	10	11
f_1	?	0	1	2	?	4	5	6	?	8	9	10
f_2	4	5	6	7	8	9	10	11	?	?	?	?
f_3	1	2	3	?	5	6	7	?	9	10	11	?
f_4	?	?	?	?	0	1	2	3	4	5	6	7
f_5	?	4	5	6	?	8	9	10	?	?	?	?
f_6	5	6	7	?	9	10	11	?	?	?	?	?
f_7	?	?	?	?	1	2	3	?	5	6	7	?
f_8	?	?	?	?	?	0	1	2	?	4	5	6

Table 1: Streaming map

Taking a look at Table 1, it seems that the cells always stay ordered. During streaming, they are all just shifted by a constant, which is different for every distribution. This is used by the PS algorithm.

1.1.2 Full PS algorithm

Let's now consider a 3 dimensional case, using the $D3Q27$ scheme with the following velocity set (Table 2):

Table 2: D3Q27 velocity set

	f_0	f_1	f_2	f_3	f_4	f_5	f_6	f_7	f_8	f_9	f_{10}	f_{11}	f_{12}	f_{13}	f_{14}	f_{15}	f_{16}	f_{17}	f_{18}	f_{19}	f_{20}	f_{21}	f_{22}	f_{23}	f_{24}	f_{25}	f_{26}
c_{qx}	0	1	−1	0	0	0	0	1	−1	1	−1	−1	1	0	0	−1	1	0	0	−1	1	−1	1	1	−1	−1	1
c_{qy}	0	0	0	0	0	−1	1	0	0	0	0	−1	1	1	−1	1	−1	1	−1	1	−1	−1	1	−1	1	−1	1
c_{qz}	0	0	0	−1	1	0	0	−1	1	1	−1	0	0	−1	1	0	0	1	−1	−1	1	1	−1	−1	1	−1	1
w	$\frac{8}{27}$	$\frac{2}{27}$	$\frac{2}{27}$	$\frac{2}{27}$	$\frac{2}{27}$	$\frac{2}{27}$	$\frac{2}{27}$	$\frac{1}{54}$	$\frac{1}{54}$	$\frac{1}{54}$	$\frac{1}{54}$	$\frac{1}{54}$	$\frac{1}{54}$	$\frac{1}{54}$	$\frac{1}{54}$	$\frac{1}{54}$	$\frac{1}{54}$	$\frac{1}{54}$	$\frac{1}{54}$	$\frac{1}{216}$	$\frac{1}{216}$	$\frac{1}{216}$	$\frac{1}{216}$	$\frac{1}{216}$	$\frac{1}{216}$	$\frac{1}{216}$	$\frac{1}{216}$

The field will be block-shaped, sized $N = N_x \cdot N_y \cdot N_z$. Cells will be indexed i, j, k to represent x, y, z position and the 3-dimensional index will be mapped onto 1D index idx as

$$idx(i, j, k) = k \cdot (N_x \cdot N_y) + j \cdot N_x + i \tag{1.1}$$

The index idx determines placement of data for each distribution array of length N in memory. To read distribution f_1 in cell i, j, k we call $f_1 = \underline{f_1}[idx(i, j, k)]$.

During the streaming step, distributions should flow into neighbour cells in directions determined by the velocity set.

A distribution value located at $idx(i, j, k)$ after streaming was placed at $idx(i - c_{qx}, j - c_{qy}, k - c_{qz})$ before streaming. This gives, independently for each $\underline{f_q}$,

$$\begin{gathered} idx(i - c_{qx}, j - c_{qy}, k - c_{qz}) \\ = \\ (k - c_{qz}) \cdot (N_x N_y) + (j - c_{qy}) \cdot N_x + (i - c_{qx}) \\ = \\ k \cdot (N_x N_y) + j \cdot N_x + i - [c_{qz} \cdot (N_x N_y) + c_{qy} \cdot N_x + c_{qx}] \\ = \\ idx(i, j, k) - s_q \end{gathered} \tag{1.2}$$

where the s_q term is constant and only depends on q (index of the distribution):

$$s_q = c_{qz} \cdot (N_x N_y) + c_{qy} \cdot N_x + c_{qx} \tag{1.3}$$

For t streaming steps,

$$idx(i - tc_{qx}, j - tc_{qy}, k - tc_{qz}) = idx(i, j, k) - ts_q \tag{1.4}$$

Instead of moving the data, it is possible to just read and write it into its original location everytime it is accessed. This can be formalized by making the index idx a function of iteration number t:

$$idx(i, j, k, t) = idx(i, j, k) - ts_q \tag{1.5}$$

To avoid negative indexes or indexes higher than N, just do

$$idx(i, j, k, t) = ((idx(i, j, k) - ts_q) \mod N + N) \mod N \tag{1.6}$$

Note that this generally makes distributions that enter the field from the boundaries invalid, but these would have to be fixed by boundary conditions anyway. There is an exception for the z axis explained in the later chapter.

When implemented, this results in by 27 parts coalesced memory access during the collision step. There is only a single large jump of the pointer per every distribution array. There is no memory access during the streaming step, except for some implementation of the index shifter. There is no second field required, and there is no difference between odd and even iterations.

1.1.3 Note about periodic boundary condition

Periodic shift streaming implicitly provides a periodic boundary condition along the z axis. This directly follows from combining (1.1), (1.6) and (1.3).

Let's look at a boundary cell in the first z layer with $k = 0$ and some general i, j indexes. At time $t = 1$, data is read from adress idx determined by equation (1.6):

$$idx(i, j, 0, 1) = ((idx(i, j, 0) - 1 \cdot s_q) \mod N + N) \mod N \tag{1.7}$$

where $idx(i, j, 0)$ is given by (1.1):

$$idx(i, j, 0) = j \cdot N_x + i \tag{1.8}$$

Looking at equation (1.3), c_{qz} can be set to 1 as those are the distributions of our interest, flowing into the boundary $z = 0$ from the left,

$$s_q = N_x N_y + c_{qy} \cdot N_x + c_{qx} \tag{1.9}$$

Combined together,

$$idx(i, j, 0, 1) = ((j \cdot N_x + i - N_x N_y - c_{qy} \cdot N_x - c_{qx}) \mod N + N) \mod N \tag{1.10}$$

Expecting $N >> N_x N_y$, the term $(j \cdot N_x + i - N_x N_y - c_{qy} \cdot N_x - c_{qx})$ will stay between $-N$ and N. This means that the innermost modulo operator in (1.10) has no effect and can be removed. Then using $N = N_x N_y N_z$, (1.10) yields

$$idx(i, j, 0, 1) = ((N_z - 1) \cdot (N_x N_y) + (j - c_{qy}) \cdot N_x + (i - c_{qx})) \mod N \tag{1.11}$$

Comparing this result to (1.1), it seems that the distributions will get periodically streamed from the correct cell in layer $N_z - 1$ if the modulo operator does not alter the index. That is satisfied if

$$(N_z - 1) \cdot (N_x N_y) + (j - c_{qy}) \cdot N_x + (i - c_{qx}) < N \tag{1.12}$$

which simplifies to

$$(j - c_{qy}) \cdot N_x + (i - c_{qx}) < N_x N_y \tag{1.13}$$

Equation (1.13) will be always satisfied, if we demand that the source cells lie within x, y bounds of the domain, which is formalized as

$$0 \leq (j - c_{qy}) \leq N_y - 1 \tag{1.14}$$

and

$$0 \leq (i - c_{qx}) \leq N_x - 1 \tag{1.15}$$

This proves that the periodic shift algorithm described by (1.1), (1.6) and (1.3) implicitly provides a periodic boundary condition along the z axis for all distributions that get streamed from cells that physically lie in the last z layer of the domain $k = N_z - 1$.

2 Boundary Conditions

2.1 Moment based boundary conditions

Moment based boundary conditions in 3 dimensions are investigated in [5], [2]. The idea is to approximate raw moments as their Maxwell-Boltzman equilibrium and use those in a system of equations to calculate the unknown distributions. The equilibriums are calculated from ρ, u_x, u_y and u_z. It is required that those 4 quantities are known a priori by prescribing them or by restoring them from known distributions. A general method to restore macroscopic quantities from known distributions is shown later on.

One advantage of using moment based boundary conditions is locality [5]. It is only necessary to work with information from a single cell.

Another advantage is that the prescribed quantities have clearer physical meaning compared to some other options, where part of the definition is somewhat artificial [5].

Derivation of moment based boundary conditions is explained below.

2.1.1 General procedure

Raw moments are defined as [1]

$$m_{abc} = \sum_i f_i c_{xi}^a c_{yi}^b c_{zi}^c = \sum_{\text{known } i} f_i c_{xi}^a c_{yi}^b c_{zi}^c + \sum_{\text{unknown } i} f_i c_{xi}^a c_{yi}^b c_{zi}^c \tag{2.1}$$

Each raw moment is just a linear combination of distributions f, from which some are known and some are unknown.
Equilibrium moments can be expressed as

$$m_{abc}^{eq} = \sum_i f_i^{eq} c_{xi}^a c_{yi}^b c_{zi}^c \tag{2.2}$$

where f_i^{eq} is the Maxwell-Boltzman equilibrium:

$$f_i^{\text{eq}} = w_i \rho \left[1 + 3\, \underline{c}_i \cdot \underline{u} + \frac{9}{2} \left(\underline{c}_i \cdot \underline{u} \right)^2 - \frac{3}{2}\, \underline{u} \cdot \underline{u} \right]. \tag{2.3}$$

Maxwell-Boltzman equilibrium moments relevant to the task are shown in Table 3. For use in a later chapter, there is also a linearization shown where any members with second or higher power of u are neglected, which is reasonable if Ma $<< 1$.

Table 3: Maxwell–Boltzmann equilibrium moments and linearized equilibrium

Moment	**Maxwell–Boltzmann equilibrium**	**Linearized equilibrium**
m_{000}	ρ	ρ
m_{100}	$\rho\, u_x$	$\rho\, u_x$
m_{010}	$\rho\, u_y$	$\rho\, u_y$
m_{001}	$\rho\, u_z$	$\rho\, u_z$
m_{200}	$\frac{1}{3}\rho + \rho\, u_x^2$	$\frac{1}{3}\rho$
m_{020}	$\frac{1}{3}\rho + \rho\, u_y^2$	$\frac{1}{3}\rho$
m_{002}	$\frac{1}{3}\rho + \rho\, u_z^2$	$\frac{1}{3}\rho$
m_{011}	$\rho\, u_y u_z$	0
m_{101}	$\rho\, u_x u_z$	0
m_{110}	$\rho\, u_x u_y$	0
m_{111}	0	0
m_{210}	$\frac{1}{3}\rho\, u_y + \rho\, u_x^2 u_y$	$\frac{1}{3}\rho\, u_y$
m_{201}	$\frac{1}{3}\rho\, u_z + \rho\, u_x^2 u_z$	$\frac{1}{3}\rho\, u_z$
m_{120}	$\frac{1}{3}\rho\, u_x + \rho\, u_x u_y^2$	$\frac{1}{3}\rho\, u_x$
m_{021}	$\frac{1}{3}\rho\, u_z + \rho\, u_y^2 u_z$	$\frac{1}{3}\rho\, u_z$
m_{102}	$\frac{1}{3}\rho\, u_x + \rho\, u_x u_z^2$	$\frac{1}{3}\rho\, u_x$
m_{012}	$\frac{1}{3}\rho\, u_y + \rho\, u_y u_z^2$	$\frac{1}{3}\rho\, u_y$
m_{022}	$\frac{1}{9}\rho + \frac{1}{3}\rho\, u_y^2 + \frac{1}{3}\rho\, u_z^2 + \rho\, u_y^2 u_z^2$	$\frac{1}{9}\rho$
m_{202}	$\frac{1}{9}\rho + \frac{1}{3}\rho\, u_x^2 + \frac{1}{3}\rho\, u_z^2 + \rho\, u_x^2 u_z^2$	$\frac{1}{9}\rho$
m_{220}	$\frac{1}{9}\rho + \frac{1}{3}\rho\, u_x^2 + \frac{1}{3}\rho\, u_y^2 + \rho\, u_x^2 u_y^2$	$\frac{1}{9}\rho$
m_{211}	$\frac{1}{3}\rho\, u_y u_z$	0
m_{121}	$\frac{1}{3}\rho\, u_x u_z$	0
m_{112}	$\frac{1}{3}\rho\, u_x u_y$	0

It is required that ρ, u_x, u_y, u_z are defined a priori, then all equilibrium moments can be calculated.

The idea is to set raw moments from equation (2.1) equal to their Maxwell-Boltzman equilibrium from Table 3. This gives a system of linear equations that allows to find unknown distributions:

$$\sum_{\text{known } i} f_i c_{xi}^a c_{yi}^b c_{zi}^c + \sum_{\text{unknown } i} f_i c_{xi}^a c_{yi}^b c_{zi}^c = m_{abc}^{eq} \tag{2.4}$$

Approximating the moments as their equilibrium is exact for raw moments up to first order. For higher order moments, using the equilibrium means that the non-equilibrium part is neglected. This approximation still recovers incompressible Navier-Stokes equations correctly [2].

The specific formulation of equations depends on type and orientation of the cell. A face cell is a cell that does not have neighbours in one direction. A edge cell does not have neighbours in two directions. A corner cell misses neighbours in three directions. A general approach to the derivation process is needed to handle all different kinds of cells [2]. A section showing the equations for one of the cell orientations as an example, follows. The procedure presented below can be algorithmized and used to automatically generate code for all different cell types.

2.1.2 Corner cell example

As an example to explain the procedure, let's use the most "difficult" case of a corner cell with outer normal $(1, -1, 1)$ (so it has neighbours in X direction but misses neighbours in positive Y and negative Z). After streaming, the distributions with $c_x = -1, c_y = 1$ or $c_z = -1$ are unknown.

Let's put all known distributions into a vector $\underline{f_k}$ and unknown distributions into $\underline{f_u}$ as displayed in Table 4.

Table 4: Known and unknown distributions for a cell with $(1, -1, 1)$ outer normal

Known distributions	$\underline{f_k} = (f_0, f_1, f_4, f_5, f_9, f_{14}, f_{16}, f_{20})^T$
Unknown distributions	$\underline{f_u} = (f_2, f_3, f_6, f_7, f_8, f_{10}, f_{11}, f_{12}, f_{13}, f_{15}, f_{17}, f_{18}, f_{19}, f_{21}, f_{22}, f_{23}, f_{24}, f_{25}, f_{26})^T$

Equilibrium moments will be stored in a vector $\underline{m} \in \mathbb{R}^{23}$:

Table 5: Vector of equilibrium moments

Equilibrium moments	$\underline{m} = \begin{pmatrix} m_{000},\ m_{100},\ m_{010},\ m_{001}, \\ m_{200},\ m_{020},\ m_{002},\ m_{011},\ m_{101},\ m_{110}, \\ m_{111}, m_{210},\ m_{201},\ m_{120},\ m_{021},\ m_{102},\ m_{012}, \\ m_{022},\ m_{202},\ m_{220},\ m_{211},\ m_{121},\ m_{112} \end{pmatrix}^T$

This allows to express equation 2.4 in a matrix form:

$$\underline{\underline{M_{fk}}}\ \underline{f_k} + \underline{\underline{M_{fu}}}\ \underline{f_u} = \underline{m} \tag{2.5}$$

where $\underline{\underline{M_{fk}}} \in \mathbb{Z}^{23 \times \mathrm{len}(\underline{f_k})}$ is the matrix of f_k coefficients:

$$\underline{\underline{M_{fk}}} = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 0 & 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & -1 & 0 & -1 & -1 & -1 \\ 0 & 0 & 1 & 0 & 1 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & -1 & 0 & -1 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & -1 & -1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 \\ 0 & 0 & 0 & 0 & 0 & 0 & -1 & -1 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & -1 & 0 & -1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 \end{bmatrix} \tag{2.6}$$

and $\underline{\underline{M_{fu}}} \in \mathbb{Z}^{23\times \mathrm{len}(\underline{f_u})}$ is the matrix of f_u coefficients:

$$\underline{\underline{M_{fu}}} = \begin{bmatrix}
1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\
-1 & 0 & 0 & 1 & -1 & -1 & -1 & 1 & 0 & -1 & 0 & 0 & -1 & -1 & 1 & 1 & -1 & -1 & 1 \\
0 & 0 & 1 & 0 & 0 & 0 & -1 & 1 & 1 & 1 & 1 & -1 & 1 & -1 & 1 & -1 & 1 & -1 & 1 \\
0 & -1 & 0 & -1 & 1 & -1 & 0 & 0 & -1 & 0 & 1 & -1 & -1 & 1 & -1 & -1 & 1 & -1 & 1 \\
1 & 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 & 1 & 0 & 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\
0 & 0 & 1 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\
0 & 1 & 0 & 1 & 1 & 1 & 0 & 0 & 1 & 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 & 0 & 1 & 1 & -1 & -1 & -1 & 1 & 1 & 1 & 1 \\
0 & 0 & 0 & -1 & -1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & -1 & -1 & -1 & -1 & 1 & 1 \\
0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & -1 & 0 & 0 & -1 & 1 & 1 & -1 & -1 & 1 & 1 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & -1 & 1 & -1 & -1 & 1 \\
0 & 0 & 0 & 0 & 0 & 0 & -1 & 1 & 0 & 1 & 0 & 0 & 1 & -1 & 1 & -1 & 1 & -1 & 1 \\
0 & 0 & 0 & -1 & 1 & -1 & 0 & 0 & 0 & 0 & 0 & 0 & -1 & 1 & -1 & -1 & 1 & -1 & 1 \\
0 & 0 & 0 & 0 & 0 & 0 & -1 & 1 & 0 & -1 & 0 & 0 & -1 & -1 & 1 & 1 & -1 & -1 & 1 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 & 0 & 1 & -1 & -1 & 1 & -1 & -1 & 1 & -1 & 1 \\
0 & 0 & 0 & 1 & -1 & -1 & 0 & 0 & 0 & 0 & 0 & 0 & -1 & -1 & 1 & 1 & -1 & -1 & 1 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & -1 & 1 & -1 & 1 & -1 & 1 & -1 & 1 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\
0 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\
0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 1 & 0 & 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 & -1 & -1 & 1 & 1 & 1 & 1 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & -1 & -1 & -1 & -1 & 1 & 1 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 & 1 & 1 & -1 & -1 & 1 & 1
\end{bmatrix} \tag{2.7}$$

Equation (2.5) gives 23 scalar equations. For a corner cell with outer normal $(1, -1, 1)$ there are 19 unknown distributions, so from (2.5) it is necessary to pick 19 equations that have linearly independent rows of $\underline{\underline{M_{fu}}}$. It is suggested to select those corresponding to lower order moments, as they have clearer physical meaning [5].

To select the equations, let's define a selector matrix $\underline{\underline{S}} \in \mathbb{Z}^{\mathrm{len}(\underline{f_u})\times 23}$. $\underline{\underline{S}}$ looks almost like an identity matrix, but 4 rows are left out to only keep the desired 19 equations:

$$\underline{\underline{S}} = \begin{bmatrix}
1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0
\end{bmatrix} \tag{2.8}$$

By multiplying (2.5) by $\underline{\underline{S}}$, the correct 19 equations are selected:

$$\underline{\underline{S}}\underline{\underline{M_{fk}}}\underline{f_k} + \underline{\underline{S}}\underline{\underline{M_{fu}}}\underline{f_u} = \underline{\underline{S}}\underline{m} \tag{2.9}$$

$\left(\underline{\underline{S}}\underline{\underline{M_{fu}}}\right)$ is regular, so it can be inverted. This allows to find $\underline{f_u}$:

$$\underline{f_u} = \left(\underline{\underline{S}}\underline{\underline{M_{fu}}}\right)^{-1} \left(\underline{\underline{S}}\underline{m} - \underline{\underline{S}}\underline{\underline{M_{fk}}}\underline{f_k}\right) \tag{2.10}$$

As $\left(\underline{\underline{S}}\ \underline{\underline{M_{fu}}}\right)$ is relatively small and nice, it can be inversed using a symbolic solver.

$$\left(\underline{\underline{S}}\ \underline{\underline{M_{fu}}}\right)^{-1} =$$

$$\begin{bmatrix}
1 & 0 & 0 & 0 & 0 & -1 & -1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\
1 & 0 & 0 & 0 & -1 & -1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\
1 & 0 & 0 & 0 & -1 & 0 & -1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\
0 & 1/2 & 0 & 0 & 1/2 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1/2 & 0 & 0 & 0 & 0 & 0 & -1/2 \\
0 & 0 & 0 & 1/2 & 0 & 0 & 1/2 & 0 & 0 & 0 & 0 & 0 & 0 & -1/2 & 0 & 0 & -1/2 & 0 & 0 \\
-1 & -1/2 & 0 & -1/2 & 1/2 & 1 & 1/2 & 0 & 0 & 0 & 0 & 0 & 1/2 & 1/2 & 0 & 0 & -1/2 & 0 & -1/2 \\
0 & 0 & -1/2 & 0 & 0 & 1/2 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1/2 & -1/2 & 0 & 0 \\
0 & 1/2 & 0 & 0 & 1/2 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1/2 & 0 & 0 & -1/2 & 0 \\
-1 & 0 & 1/2 & -1/2 & 1 & 1/2 & 1/2 & 0 & 0 & 0 & -1/2 & 1/2 & 0 & 0 & 0 & 0 & 0 & -1/2 & -1/2 \\
-1 & -1/2 & 1/2 & 0 & 1/2 & 1/2 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1/2 & -1/2 & -1/2 & -1/2 & 0 \\
0 & 0 & 0 & 1/2 & 0 & 0 & 1/2 & 0 & 0 & 0 & 0 & -1/2 & 0 & 0 & 0 & 0 & 0 & -1/2 & 0 \\
0 & 0 & -1/2 & 0 & 0 & 1/2 & 0 & 0 & 0 & 0 & 1/2 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1/2 \\
1 & 1/2 & -1/2 & 1/2 & -1/2 & -1/2 & -1/2 & -1/4 & 1/4 & -1/4 & 1/4 & -1/4 & -1/4 & -1/4 & -1/4 & 1/4 & 1/4 & 1/4 & 1/4 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & -1/4 & 0 & 0 & 0 & 0 & 0 & 1/4 & 0 & -1/4 & 1/4 & 0 & 0 \\
0 & -1/2 & 0 & 0 & -1/2 & 0 & 0 & 0 & -1/4 & 1/4 & 1/4 & -1/4 & 1/4 & 0 & 1/4 & 0 & 0 & 1/4 & 1/4 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1/4 & -1/4 & 0 & 1/4 & 0 & 0 & 0 & 0 & 0 & 1/4 \\
0 & 0 & 0 & -1/2 & 0 & 0 & -1/2 & 1/4 & -1/4 & 0 & 0 & 1/4 & 0 & 1/4 & -1/4 & 1/4 & 1/4 & 1/4 & 0 \\
0 & 0 & 1/2 & 0 & 0 & -1/2 & 0 & 1/4 & 0 & 1/4 & -1/4 & 0 & -1/4 & -1/4 & 0 & -1/4 & 1/4 & 0 & 1/4 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1/4 & 0 & 0 & 1/4 & 0 & 0 & 1/4 & 0 & 0 & 1/4 & 0
\end{bmatrix} \tag{2.11}$$

2.2 Reconstruction of macroscopic quantities from known distributions

The moment based boundary conditions allow to calculate $\underline{f_u}$ from $\underline{f_k}$ and macroscopic quantities ρ, u_x, u_y, u_z. In case that not all macroscopic quantities are explicitly prescribed, it is possible to reconstruct them from $\underline{f_k}$.

2.2.1 Corner cell example with unknown ρ, u_x, u_y, u_z

Let's consider a corner cell with $(1, -1, 1)$ outer normal again. Let's first illustrate a case where no macroscopic quantity is prescribed, to show how it is possible to reconstruct all ρ, u_x, u_y, u_z just from $\underline{f_k}$.

Known and unknown distributions for such case were previously displayed in Table 4. Matrices $\underline{\underline{M_{fk}}}$ and $\underline{\underline{M_{fu}}}$ were listed in equations (2.6) and (2.7), respectively.

In the previous chapter, the goal was to calculate $\underline{f_u}$, which required finding linearly independeng rows of $\underline{\underline{M_{fu}}}$. Now the situation is reversed: goal is to find at least 4 independent equations for ρ, u_x, u_y, u_z where no unknown f_i is present.

This can be achieved by forming linear combinations of the available equations. Let $\underline{c} \in \mathbb{R}^{23}$ denote a column vector of coefficients, where each component multiplies the corresponding equation. Then the unknown distributions get eliminated if for all $\underline{f_u}$ it holds that

$$\left(\underline{\underline{M_{fu}}}\ \underline{f_u}\right) \cdot \underline{c} = 0 \tag{2.12}$$

This can be reformulated as

$$\left(\underline{\underline{M_{fu}}}\ \underline{f_u}\right) \cdot \underline{c} = \underline{c}^T \left(\underline{\underline{M_{fu}}}\ \underline{f_u}\right) = \left(\underline{c}^T\ \underline{\underline{M_{fu}}}\right) \underline{f_u} = \underline{f_u}^T \left(\underline{c}^T\ \underline{\underline{M_{fu}}}\right)^T = \underline{f_u}^T \left(\underline{\underline{M_{fu}}}^T\ \underline{c}\right) = 0 \tag{2.13}$$

which will hold for all $\underline{f_u}$ only if

$$\underline{\underline{M_{fu}}}^T\ \underline{c} = \underline{0} \tag{2.14}$$

Therefore, the searched coefficient vectors $\underline{c}$ are given by the null space of $\underline{\underline{M_{fu}}}^T$. Let's assemble all n linearly independent vectors $\underline{c}$ into a matrix $\underline{\underline{C}}$:

$$\underline{\underline{C}} = \begin{bmatrix} | & | & | & | \\ \underline{c}_0 & \underline{c}_1 & \dots & \underline{c}_n \\ | & | & | & | \end{bmatrix} \tag{2.15}$$

where $\underline{c_0}, \underline{c_1} \dots$ are ordered smallest to largest using the key

$$\text{key} = \sum_i^{\text{len}(\underline{c})} w_i, \quad w_i = \begin{cases} \text{order of moment } m_i \text{ if } c_i \neq 0 \\ 0 \text{ otherwise} \end{cases} \tag{2.16}$$

which puts low order moment combinations first. In case of outer normal $(1, -1, 1)$, $\underline{\underline{C}}$ yields

$$\underline{\underline{C}} = \begin{bmatrix} -1 & 1 & -1 & 1 \\ -1 & 0 & -1 & 1 \\ 1 & -1 & 0 & -1 \\ -1 & 1 & -1 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \\ 1 & -1 & 0 & 0 \\ -1 & 0 & -1 & 0 \\ 1 & 0 & 0 & -1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{2.17}$$

That suggests how to deal with f_u. To relate remaining f_k to ρ, u_x, u_y, u_z, equilibrium moments will be used once again. However, Maxwell-Boltzman equilibrium moments as shown in Table 3 are nonlinear. Therefore the linearized equilibrium will be used to provide a linear set of equations.

Let's define a vector of independent macroscopic quantities $\underline{q}$:

$$\underline{q} = (\rho, \rho u_x, \rho u_y, \rho u_z)^T \tag{2.18}$$

Then linearized equilibrium moments can be expressed as

$$\underline{m} = \underline{\underline{Q}}\ \underline{q} \tag{2.19}$$

where $\underline{\underline{Q}} \in \mathbb{Z}^{23\times 4}$:

$$\underline{\underline{Q}} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ \frac{1}{3} & 0 & 0 & 0 \\ \frac{1}{3} & 0 & 0 & 0 \\ \frac{1}{3} & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & \frac{1}{3} & 0 \\ 0 & 0 & 0 & \frac{1}{3} \\ 0 & \frac{1}{3} & 0 & 0 \\ 0 & 0 & 0 & \frac{1}{3} \\ 0 & \frac{1}{3} & 0 & 0 \\ 0 & 0 & \frac{1}{3} & 0 \\ \frac{1}{9} & 0 & 0 & 0 \\ \frac{1}{9} & 0 & 0 & 0 \\ \frac{1}{9} & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \tag{2.20}$$

Now, inserting (2.19) into (2.5) and multiplying both sides from the left by $\underline{\underline{C}}^T$ gives

$$\underline{\underline{C}}^T \left(\underline{\underline{M_{fk}}} \; \underline{f_k} + \underline{\underline{M_{fu}}} \; \underline{f_u} \right) = \underline{\underline{C}}^T \; \underline{\underline{Q}} \; \underline{q} \tag{2.21}$$

As follows from above, $\underline{\underline{C}}^T$ is built so that $\underline{f_u}$ gets cancelled, so

$$\underline{\underline{C}}^T \underline{\underline{M_{fk}}} \; \underline{f_k} = \underline{\underline{C}}^T \; \underline{\underline{Q}} \; \underline{q} \tag{2.22}$$

Equation (2.22) provides n scalar equations for n vectors $\underline{c}$. The unknown is $\underline{q}$. In a general case it may happen that n is higher than 4 which is the number of unknowns. Therefore we need to select such equations that have linearly independent rows of $\left(\underline{\underline{C}}^T \; \underline{\underline{Q}} \right)$ and which use the lowest order moments. Thanks to the ordering (2.16), the preferred rows are placed on top.

To select the equations, let's use a new selector matrix $\underline{\underline{S_q}} \in \mathbb{Z}^{4 \times \mathrm{columns}(\underline{C})}$. In case of outer normal $(1, -1, 1)$, $\underline{\underline{S}}$ yields:

$$\underline{\underline{S_q}} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{2.23}$$

Multiplying (2.22) by $\underline{\underline{S_q}}$ selects the correct equations:

$$\underline{\underline{S_q}} \underline{\underline{C}}^T \underline{\underline{M_{fk}}} \; \underline{f_k} = \underline{\underline{S_q}} \underline{\underline{C}}^T \underline{\underline{Q}} \; \underline{q} \tag{2.24}$$

where $\left(\underline{\underline{S_q}} \underline{\underline{C}}^T \underline{\underline{Q}} \right)$ is regular, so it can be inverted. This allows to find $\underline{q}$:

$$\underline{q} = \left(\underline{\underline{S_q}} \underline{\underline{C}}^T \underline{\underline{Q}} \right)^{-1} \underline{\underline{S_q}} \underline{\underline{C}}^T \underline{\underline{M_{fk}}} \; \underline{f_k} \tag{2.25}$$

As $\left(\underline{\underline{S_q}}\underline{\underline{C}}^T\underline{\underline{Q}}\right)$ is relatively small and nice, it can be inversed using a symbolic solver.

$$\left(\underline{\underline{S_q}}\underline{\underline{C}}^T\underline{\underline{Q}}\right)^{-1} = \begin{bmatrix} 2 & 3/2 & -3/2 & 3/2 \\ -1 & -3/2 & 0 & 0 \\ 1 & 0 & -3/2 & 0 \\ -1 & 0 & 0 & -3/2 \end{bmatrix} \tag{2.26}$$

2.2.2 Corner cell example with prescribed ρ or u_x, u_y, u_z

Let's modify the procedure for a case when either ρ is given or u_x, u_y, u_z are given. Compared to the previous derivation, $\underline{f_k}, \underline{f_u}, \underline{\underline{M_{fk}}}, \underline{\underline{M_{fu}}}, \underline{\underline{C}}$ stay the same. However, the vector of independent macroscopic quantities should be split in the ρ and u part. Let's define

$$\underline{u} = (u_x, u_y, u_z)^T \tag{2.27}$$

which results in new expression of $\underline{m}$:

$$\underline{m} = \rho\left(\underline{\underline{Q_u}}\ \underline{u} + \underline{Q_\rho}\right) \tag{2.28}$$

where

$$\underline{\underline{Q_u}} = \begin{bmatrix} 0 & 0 & 0 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & \frac{1}{3} & 0 \\ 0 & 0 & \frac{1}{3} \\ \frac{1}{3} & 0 & 0 \\ 0 & 0 & \frac{1}{3} \\ \frac{1}{3} & 0 & 0 \\ 0 & \frac{1}{3} & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \tag{2.29}$$

and

$$\underline{Q_\rho} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \\ \frac{1}{3} \\ \frac{1}{3} \\ \frac{1}{3} \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ \frac{1}{9} \\ \frac{1}{9} \\ \frac{1}{9} \\ 0 \\ 0 \\ 0 \end{bmatrix} \tag{2.30}$$

That leads to

$$\underline{\underline{C}}^T \underline{\underline{M_{fk}}}\ \underline{f_k} = \underline{\underline{C}}^T \rho \left(\underline{\underline{Q_u}}\underline{u} + \underline{Q_\rho} \right) \tag{2.31}$$

reorganized

$$\frac{1}{\rho} \underline{\underline{C}}^T \underline{\underline{M_{fk}}} \underline{f_k} - \underline{\underline{C}}^T \underline{Q_\rho} = \underline{\underline{C}}^T \underline{\underline{Q_u}} \underline{u} \tag{2.32}$$

From here the case splits depending on if ρ or $\underline{u}$ is the unknown. If $\underline{u}$ is the unknown, we need to select such equations that have linearly independent rows of $\left(\underline{\underline{C}}^T \underline{\underline{Q_u}} \right)$. For that let's use a selector matrix $\underline{\underline{S_u}} \in \mathbb{Z}^{4 \times \text{columns}(\underline{\underline{C}})}$:

$$\underline{\underline{S_u}} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \tag{2.33}$$

Then

$$\underline{u} = \left(\underline{\underline{S_u}} \underline{\underline{C}}^T \underline{\underline{Q_u}} \right)^{-1} \left[\frac{1}{\rho} \underline{\underline{S_u}} \underline{\underline{C}}^T \underline{\underline{M_{fk}}} \underline{f_k} - \underline{\underline{S_u}} \underline{\underline{C}}^T \underline{Q_\rho} \right] \tag{2.34}$$

where

$$\left(\underline{\underline{S_u}} \underline{\underline{C}}^T \underline{\underline{Q_u}} \right)^{-1} = \begin{bmatrix} -1 & -3/2 & 0 \\ 1 & 0 & -3/2 \\ 1 & 3/2 & -3/2 \end{bmatrix} \tag{2.35}$$

If ρ is the unknown in (2.32), we can manipulate (2.32) to

$$\underline{\underline{C}}^T \underline{\underline{M_{fk}}} \underline{f_k} = \rho \left[\underline{\underline{C}}^T \underline{Q_\rho} + \underline{\underline{C}}^T \underline{\underline{Q_u}} \underline{u} \right] \tag{2.36}$$

To isolate ρ, we need to define $\underline{\underline{S_\rho}} \in \mathbb{Z}^{1 \times \text{columns}(\underline{\underline{C}})}$ so that it selects a single equation:

$$\underline{\underline{S_\rho}} \underline{\underline{C}}^T \underline{\underline{M_{fk}}} \underline{f_k} = \rho \left[\underline{\underline{S_\rho}} \underline{\underline{C}}^T \underline{Q_\rho} + \underline{\underline{S_\rho}} \underline{\underline{C}}^T \underline{\underline{Q_u}} \underline{u} \right] \tag{2.37}$$

Here we need to ensure that $\underline{\underline{S_\rho}}\underline{\underline{C}}^T\underline{Q_\rho} + \underline{\underline{S_\rho}}\underline{\underline{C}}^T\underline{\underline{Q_u}}\underline{u}$ is not 0. The dependency on $\underline{u}$ is uncomfortable, but by looking at size of elements in $\underline{Q_\rho}, \underline{\underline{Q_u}}$ and assuming that Ma $<<$ 1, it should be sufficient to ensure that $\underline{\underline{S_\rho}}\underline{\underline{C}}^T\underline{Q_\rho}$ is not 0. This is satisfied for the present case by

$$\underline{\underline{S_\rho}} = \begin{bmatrix} 1 & 0 & 0 & 0 \end{bmatrix} \tag{2.38}$$

Then

$$\rho = \left(\underline{\underline{S_\rho}}\underline{\underline{C}}^T\underline{Q_\rho} + \underline{\underline{S_\rho}}\underline{\underline{C}}^T\underline{\underline{Q_u}}\underline{u}\right)^{-1}\underline{\underline{S_\rho}}\underline{\underline{C}}^T\underline{\underline{M_{fk}}}\underline{f_k} \tag{2.39}$$

Note that $\left(\underline{\underline{S_\rho}}\underline{\underline{C}}^T\underline{Q_\rho} + \underline{\underline{S_\rho}}\underline{\underline{C}}^T\underline{\underline{Q_u}}\underline{u}\right)$ is a scalar, so there is no matrix inversion needed.

3 Grid refinement

A grid refinement approach will be presented that combines methods from [7] and [4].

The idea is to create a block of cells with 2× finer resolution, which results in placing 2^D fine cells in place previously occupied by one coarse cell (D is the spatial dimension). This is illustrated in Figure 3:

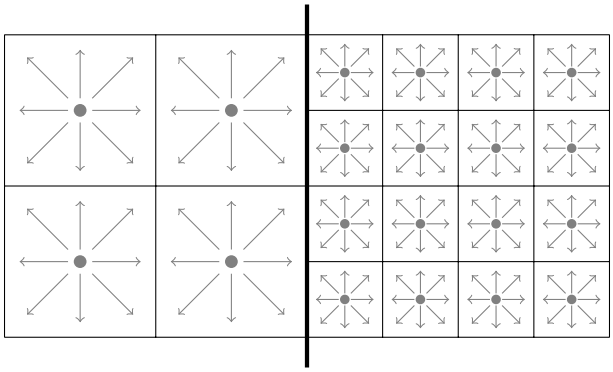

Figure 3: Block grid refinement

3.1 Adjustment of parameters on the fine grid

In LBM, we use dimensionless quantities defined as

$$x = \frac{x^{\text{phys}}}{\text{res}}, \quad x^{\text{phys}} = x \cdot \text{res} \tag{3.1}$$

$$t = \frac{t^{\text{phys}}}{dt^{\text{phys}}}, \quad t^{\text{phys}} = t \cdot dt^{\text{phys}} \tag{3.2}$$

$$u = u^{\text{phys}} \frac{dt^{\text{phys}}}{\text{res}}, \quad u^{\text{phys}} = u \frac{\text{res}}{dt^{\text{phys}}} \tag{3.3}$$

$$\nu = \nu^{\text{phys}} \frac{dt^{\text{phys}}}{\text{res}^2}, \quad \nu^{\text{phys}} = \nu \frac{\text{res}^2}{dt^{\text{phys}}} \tag{3.4}$$

where res (resolution) is physical length of one cell and dt^{phys} is physical time of one iteration.

Dimensionless kinematic viscosity ν, relaxation time τ and relaxation frequency ω are related the following way:

$$\tau = \frac{1}{\omega} = 3\nu + \frac{1}{2} \tag{3.5}$$

When switching from coarse (c) to fine grid (f), resolution becomes 2× finer:

$$\text{res}_f = \frac{\text{res}_c}{2} \tag{3.6}$$

Physical velocity must stay the same:

$$u_f^{\text{phys}} = u_c^{\text{phys}} \tag{3.7}$$

So

$$u_f \frac{\text{res}_f}{dt_f^{\text{phys}}} = u_c \frac{\text{res}_c}{dt_c^{\text{phys}}} \tag{3.8}$$

Because we want to keep the same lattice Mach number, the dimensionless velocity must stay the same, too:

$$u_f = u_c \tag{3.9}$$

Therefore

$$\frac{\text{res}_f}{dt_f^{\text{phys}}} = \frac{\text{res}_c}{dt_c^{\text{phys}}} \tag{3.10}$$

And so the physical timestep for the fine grid must be 2× shorter than for the coarse grid:

$$dt_f^{\text{phys}} = \frac{\text{res}_f}{\text{res}_c} dt_c^{\text{phys}} = \frac{1}{2} dt_c^{\text{phys}} \tag{3.11}$$

This means, for 1 step on the coarse grid, we need to run 2 steps on the fine grid.

We also set that

$$\rho_f = \rho_c \tag{3.12}$$

Next, we require that physical kinematic viscosity stays the same:

$$\nu_f^{\text{phys}} = \nu_c^{\text{phys}} \tag{3.13}$$

Which gives prescription for the lattice viscosity:

$$\nu_f = 2\nu_c \tag{3.14}$$

It is now a the main question how to rescale the distribution functions. Because $u_f = u_c$ and $\rho_f = \rho_c$ as defined above, and the equillibrium part of the distribution functions is only a function of u and ρ, values of the equillibrium distribution functions stay the same for both grids.

$$\underline{f}_f^{eq} = \underline{f}_c^{eq} \tag{3.15}$$

There have been multiple approaches to the treatment of the non-equillibrium part of the distribution functions. [7] suggests to avoid any rescaling, whereas [3] multiplies the non-equillibrium part by a constant.

Here, the central moment approach from [4] will be used. Central moments are defined as

$$k_{abc} = \sum_i f_i (c_{xi} - u_x)^a (c_{yi} - u_y)^b (c_{zi} - u_z)^c \tag{3.16}$$

Because the second order central moments $k_{011}, k_{101}, k_{110}$ are proportional to spatial derivatives of momentum and the spatial scale differs between the grids, those moments need to be rescaled when travelling across grids:

$$\begin{cases} k_{011f} = \frac{1}{2} k_{011c} \\ k_{101f} = \frac{1}{2} k_{101c} \\ k_{110f} = \frac{1}{2} k_{110c} \end{cases} \tag{3.17}$$

Similarly, the combinations of $k_{200}, k_{020}, k_{002}$ that correspond to shear stress need to be rescaled, resulting in the following scheme:

$$\begin{cases} k_{200f} + k_{020f} + k_{002f} = k_{200c} + k_{020c} + k_{002c} \\ k_{200f} - k_{020f} = \frac{1}{2} \left(k_{200c} - k_{020c} \right) \\ k_{200f} - k_{002f} = \frac{1}{2} \left(k_{200c} - k_{002c} \right) \end{cases} \tag{3.18}$$

and so

$$\begin{cases} k_{200f} = \frac{2}{3} k_{200c} + \frac{1}{6} k_{020c} + \frac{1}{6} k_{002c} \\ k_{020f} = \frac{1}{6} k_{200c} + \frac{2}{3} k_{020c} + \frac{1}{6} k_{002c} \\ k_{002f} = \frac{1}{6} k_{200c} + \frac{1}{6} k_{020c} + \frac{2}{3} k_{002c} \end{cases} \tag{3.19}$$

and

$$\begin{cases} k_{200c} = \frac{5}{3} k_{200f} - \frac{1}{3} k_{020f} - \frac{1}{3} k_{002f} \\ k_{020c} = -\frac{1}{3} k_{200f} + \frac{5}{3} k_{020f} - \frac{1}{3} k_{002f} \\ k_{002c} = -\frac{1}{3} k_{200f} - \frac{1}{3} k_{020f} + \frac{5}{3} k_{002f} \end{cases} \tag{3.20}$$

We are using cummulant collision, which involves transformation to central moments. Therefore to perform the transformation of the distribution functions when travelling between grids, we can perform the central moment transformation, scale the relevant moments, set the high order cummulants to zero, skip the relaxation part and perform the backwards transformation to receive the rescaled distribution functions on the target grid.

3.2 Communication on the grid interface

We now need an algorithm that will ensure that information travels correctly from the coarse grid into the fine grid and conversely.

Let's consider a vertical interface running along the y axis between the coarse grid (on the left) and the fine grid (on the right).

On such interface, there are some distributions that have zero c_x component and will not cross the interface during the streaming step. These can be handled by the two grids independently.

The distributions with a non-zero c_x component need to be propagated into the opposing grid during the streaming step. We will illustrate how that works on f_1 and f_3 of the D2Q9 scheme (Figure 1). The grid interface with distributions f_1 and f_3 is displayed in Figure 4.

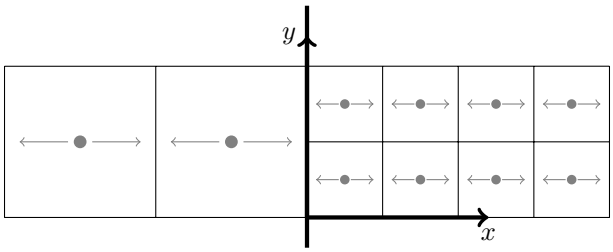

Figure 4: Grid interface

The algorithm requires that we add one layer of coarse ghost cells, and two layers of fine ghost cells on the opposite side of the interface, so that there is an overlap wide 2 coarse cells or 4 fine cells. This placement used by [7] causes that each coarse cell exactly overlaps 2^D fine cells, which guarantees mass conservation. However the overlaps are defined in a simplified way compared to [4]. The placement is illustrated in Figure 5.

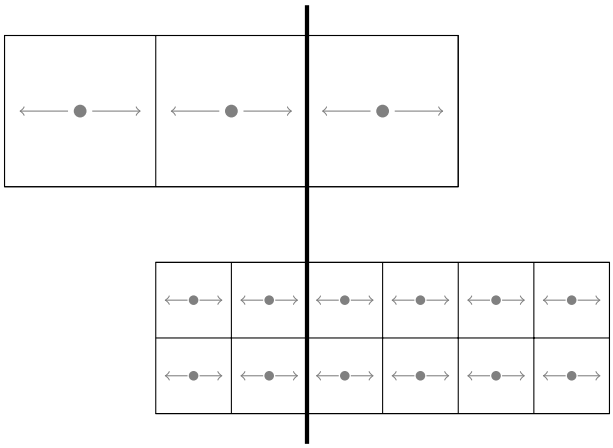

Figure 5: Ghost cells around grid interface

The algorithm consists of 4 steps.

1. **Communication from fine grid to coarse grid.** Read all f from the fine cells that lie under one coarse ghost cell. Take an average over those. Rescale f as described above. Then, write all f into the coarse ghost cell. The communication step from fine to coarse is illustrated in Figure 6.

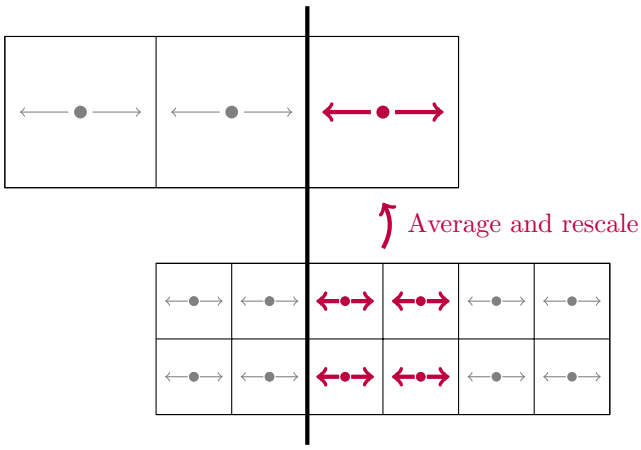

Figure 6: Communication from coarse grid to fine grid

2. **Communication from coarse grid to fine grid.** From the last non-ghost coarse cell (x, y, z), read all f. Then, read all f also from all 4 face-neighbours of that cell. Note that one of the neighbours that lies inside the refinement area is already using the information that was transferred from the fine grid. That should not be an issue, since the fine grid solution is supposed to be more accurate.
Use central differences to approximate spatial derivatives of f_q along all axes:

$$\frac{\partial f_q}{\partial x} \approx \frac{1}{2}\left(f_q(x+1,y,z) - f_q(x-1,y,z)\right) \tag{3.21}$$

$$\frac{\partial f_q}{\partial y} \approx \frac{1}{2}\left(f_q(x,y+1,z) - f_q(x,y-1,z)\right) \tag{3.22}$$

$$\frac{\partial f_q}{\partial z} \approx \frac{1}{2}\left(f_q(x,y,z+1) - f_q(x,y,z-1)\right) \tag{3.23}$$

If some of the neighbour cells don't exist, such as on the boundary of the coarse domain, use forward or backward difference for that direction:

$$\frac{\partial f_q}{\partial x} \approx f_q(x+1,y,z) - f_q(x,y,z) \tag{3.24}$$

$$\frac{\partial f_q}{\partial x} \approx f_q(x,y,z) - f_q(x-1,y,z) \tag{3.25}$$

From that, approximate f_q at the center of each fine cell:

$$f_q(x \pm \frac{1}{4}, y \pm \frac{1}{4}, z \pm \frac{1}{4}) = f_q(x,y,z) \pm \frac{1}{4}\frac{\partial f_q}{\partial x} \pm \frac{1}{4}\frac{\partial f_q}{\partial y} \pm \frac{1}{4}\frac{\partial f_q}{\partial x} \tag{3.26}$$

This interpolation is less accurate than what is presented in [4], but more accurate than [7]. Next, scale f as described above. Then, write f into all corresponding ghost cells of the fine grid (there are 2^D of them).
The communication step from coarse to fine is illustrated in Figure 7.

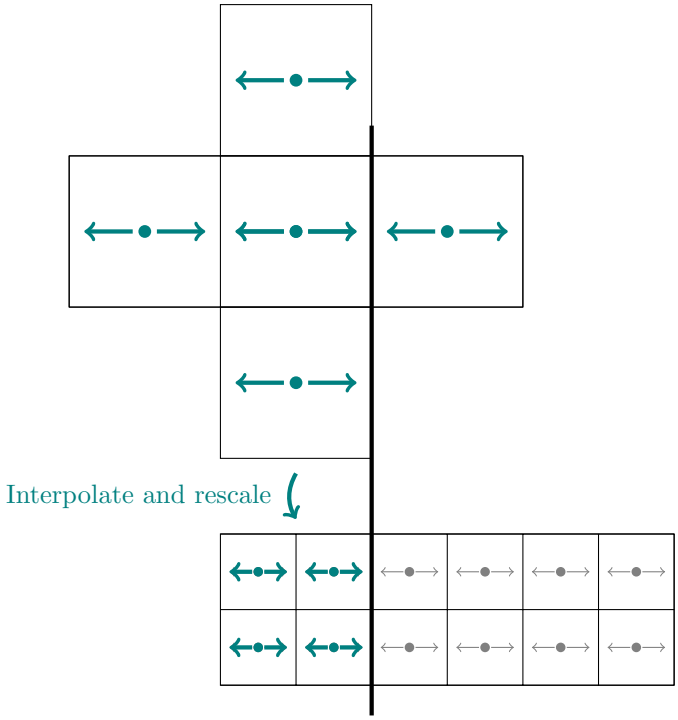

Figure 7: Communication from coarse grid to fine grid

3. **Perform one iteration on the coarse grid.** Perform streaming and collision on the coarse grid.
4. **Perform two iterations on the fine grid.** Perform streaming and collision on the fine grid two times.

4 References

[1] Alessandro De Rosis. Nonorthogonal central-moments-based lattice boltzmann scheme in three dimensions. *Phys. Rev. E*, 95:013310, Jan 2017.

[2] Pavel Eichler, Radek Fučík, and Pavel Strachota. Investigation of mesoscopic boundary conditions for lattice boltzmann method in laminar flow problems. *Computers & Mathematics with Applications*, 173:87–101, 2024.

[3] Olga Filippova and Dieter Hänel. Grid refinement for lattice-bgk models. *Journal of Computational Physics*, 147(1):219–228, 1998.

[4] M. Geier, A. Greiner, and J. G. Korvink. Bubble functions for the lattice boltzmann method and their application to grid refinement. *The European Physical Journal Special Topics*, 171(1):173–179, April 2009.

[5] Ivars Krastins, Andrew Kao, Koulis Pericleous, and Tim Reis. Moment-based boundary conditions for straight on-grid boundaries in three dimensional lattice boltzmann simulations. *International Journal for Numerical Methods in Fluids*, 92:1948–1974, 05 2020.

[6] Adrian Kummerländer, Márcio Dorn, Martin Frank, and Mathias J. Krause. Implicit propagation of directly addressed grids in lattice boltzmann methods. *Concurrency and Computation: Practice and Experience*, 35(8):e7509, 2023.

[7] M. Rohde, D. Kandhai, J. J. Derksen, and H. E. A. van den Akker. A generic, mass conservative local grid refinement technique for lattice-boltzmann schemes. *International Journal for Numerical Methods in Fluids*, 51(4):439–468, 2006.